INBRE-retina Design Document

Intent

The intent of this application is to make it possible for people (and potential future algorithm based extensions) to identify and label cells found within multidimensional videos of fish retinas. The exact details of the video formats and exact preferences/needs are still unknown but for now video viewing, basic labeling functions, and import and export capabilities are needed.

Class descriptions

Main:
Initializes the master memory and begins generating the views which will govern the UI.

Views:

MainWindowView: A class for rendering the main window and its contents. Extends qtw.QMainWindow. Contains the MainWindowMenuBar view and the CenterBox view.

WindowMenuBarView: A class for rendering the menubar for the main window. Currently most buttons are placeholders. Should provide UI options to the user such as “Open file”, “Export Labels”, “Import Labels”, “Settings” etc.

CenterBox: A container for other views to be rendered within the greater main window. Sets some basic layout settings and contains ControlAreaView and MediaAreaView.

ControlsAreaView: A container for the different controls the user may need to view and manipulate the labels on the canvas. Contains MainControlView, and LabelDataView.

MainControlsView: Containing mostly temporary code at this time. Intended to hold controls the user can access such as changing the current label interaction mode (select label, draw label, erase label).

LabelDataView: Old probably unnecessary code, needs refactor or removal. Intended to contain and display fields which contained the data for any label actively being viewed by the user. Possibly also would allow the user to edit any data there in case of error. Currently does nothing but show some old dummy fields.

MediaAreaView: Likely unnecessary code. Serves as a container for CanvasView and nothing else at this time, though it does set some layout rules. May also contain video controls in the future.

CanvasView: extends QGraphicsView and serves to both render the current image but also receives user input when selecting a bounding box or drawing a new one. Data for drawing new bounding boxes originates from here. Also instantiates LabelPopup though that may change.

LabelPopup: possibly unnecessary code and currently in need of refactoring. The intended purpose was to make it so when a user drew a new bounding box for a label a popup would appear with fields the user could fill in with all needed data for the label. (i.e. what cell(s) are included in the bounding box, what event(s) are included, wether these are the first instances of a cell or event, etc.)

Presenters:

AbstractPresenter: Defines some minimum requirements for other presenters such as requiring a view class gets defined. Also provides a getView method.

CanvasPresenter: Facilitates communication between the canvas view the user interacts with and the canvas model which actually holds the image data and accesses the label data.

LabelDataPresenter: Facilitates communication between the possibly unneeded label data view fields and the LabelData object which holds all relevant Label data for the file.

LabelPopupPresenter: Facilitates communication between the label popup views user interface and its model which handles sending new data to the LabelDataModel.

MainControlsPresenter: Facilitates communication between the main controls and master memory. At this time only sets the users interaction mode. This code may need refactoring.

WindowMenuBarPresenter:Facilitates communication between the MenuBars view and its model.

Models:

MasterMemory: A class that holds essential application data as class wide variables to ensure all other classes can access up to data data. Holds such data as the labels that apply to the given file, the current canvas, and the current user interaction mode.

LabelData: A class which defines how frame data, bounding box data, cell data, event data, type data, and file metadata are stored. Everything at the moment is stored as dictionaries or lists in the hope of making exporting the data easier.
LabelExporter: Does nothing yet. Intended to be a class which takes Label Data from master memory and encodes it in JSON (or whatever other format the project desires). The exporter then saves the data on the computer so it can be used later.

LabelImporter: Does nothing yet. Intended to take data previously encoded using the exporter and convert it back into a LabelData object so the data in it can be rendered and manipulated by the user.

LabelPopupModel:An out of date class I will probably remove soon. It grabs data from Label data and attempts to push new label data to it. It was an attempt to implement the users ability to push new labels to the masterMemory. It does not work at this time.

nd2FileAccessor: Out of date code. An early attempt to access parts of nd2 multi layered video files without loading them all into memory. Didnt work and will probably need to be removed.

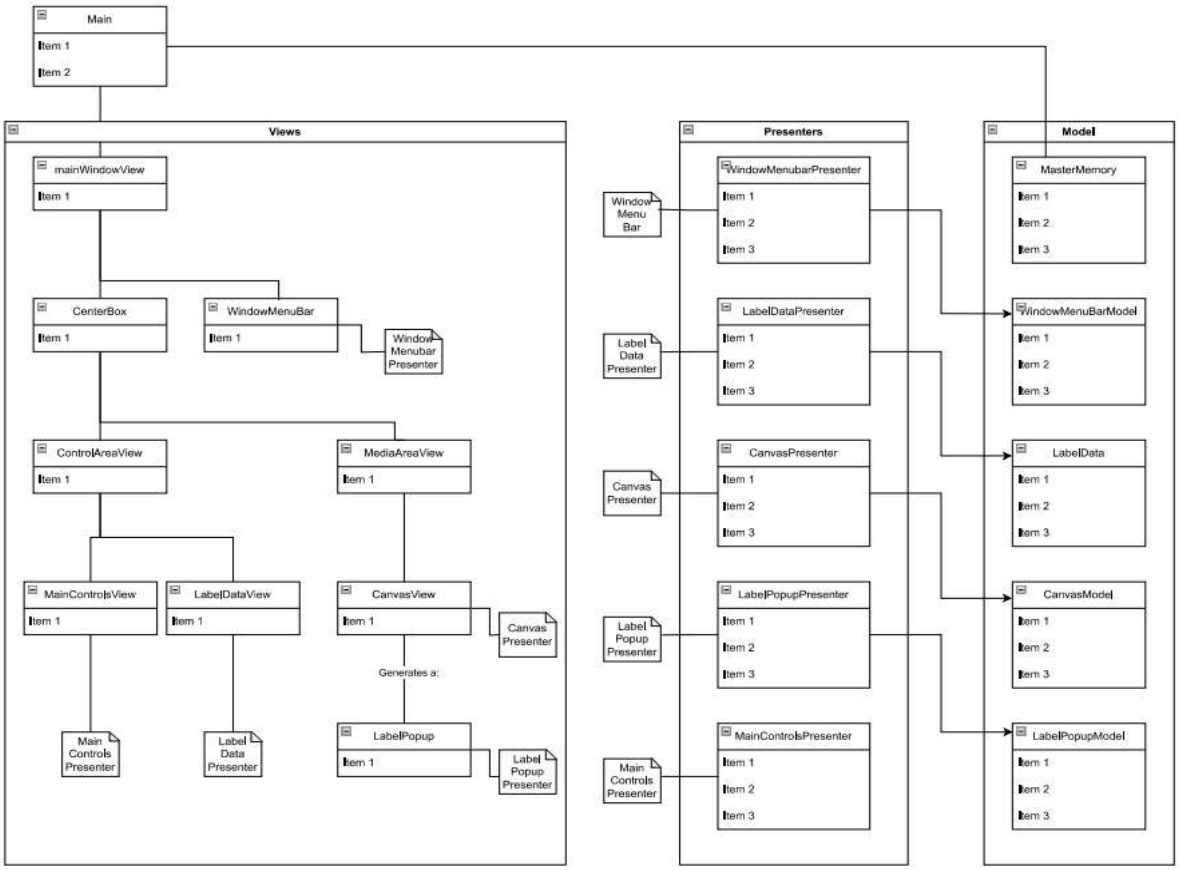
WindowMenuBarModel: The model for the main views menubar. At this time just opens a single frame from a small selection of compatible formats. Final intention is for it to perform all logic necessary for the WindowMenuBarView to function i.e. open a file, export a file, import a file, change app settings, etc.

Displayable: An abstract class ensuring that any video or image format a developer wants to make compatible with the application must have a getPixmap method which returns a pixmap.

SimpleMovie: Extends displayable. Accepts GIF, MNG, and APNG formats and allows for extracting individual frames as pixmaps.

StandardImage: Does nothing at this time. Intended to let the user just render an image as a pixmap in case someone wishes to label cells on one image rather than a video. Will extend Displayable

UML



Note: this unfinished UML is just a placeholder while I work on finishing it.